Estação distribuída de contenção de enchentes

Vitor Henrique Ribeiro Matias
github.com/Vitorhrm/estacao-enchentes-sd

Protótipo de monitoramento e controle com MQTT, XML-RPC e quatro controladores replicados.

Problema e justificativa

Em um reservatório urbano, uma chuva intensa pode elevar o nível rapidamente. A bomba precisa responder a essa mudança e respeitar a posição da comporta. A queda de um controlador pode atrasar o acionamento e uma resposta errada pode autorizar uma operação incompatível com a condição simulada.

Objetivo

Monitorar três sinais em tempo quase real e permitir que um operador altere o modo da bomba, com confirmação de aplicação e tolerância a uma falha de controlador por rodada.

Política de controle

No modo AUTO, com a comporta aberta, a bomba liga a partir de 100 cm. Chuva de pelo menos 35 mm/h antecipa a partida para 80 cm. Depois de ligada, a bomba se mantém acima de 60 cm. A diferença entre os limiares de partida e parada evita comutações frequentes perto do limite.

Modos e intertravamento

MANUAL_OFF solicita desligamento. MANUAL_ON solicita acionamento e respeita a comporta. Com a comporta fechada ou telemetria incompleta ou vencida, os controladores votam por desligar. A política usa limiares didáticos e a bomba é simulada.

Plano realizado

Modelagem dos sinais → integração dos processos → persistência e quórum → injeção de falhas → documentação e publicação.

Arquitetura e chamada remota

Cada papel executa em um processo. O broker encaminha mensagens entre os componentes e o cliente usa um objeto remoto.

Cliente ↔ middleware · XML-RPC

FloodStation expõe status() e set_mode(modo, id). O operador pode consultar continuamente ou alterar o modo pelo terminal.

Sensores → broker → middleware · MQTT

Um simulador publica nível, chuva e posição da comporta em sensor/{campo}. O middleware valida autoria, tipo, faixa e idade.

Middleware ↔ broker ↔ quatro controladores · MQTT

A rodada sai em round/{nó}. Cada controlador grava entrada e voto no SQLite local e responde em vote.

Middleware ↔ broker ↔ atuador · MQTT

command solicita a aplicação. ack confirma a gravação. probe e state consultam o estado atual e recibos por ID.

Fluxo de set_mode()

O middleware consulta o atuador e congela as entradas da rodada. As réplicas avaliam o mesmo modo, a mesma telemetria e o mesmo estado anterior. Com três votos coincidentes, o middleware envia o comando. O atuador registra modo e estado e publica a confirmação vinculada à rodada e ao ID.

Acompanhamento e automação

O cliente consulta status() a cada segundo no modo acompanhar. O middleware também verifica periodicamente a regra de controle e abre uma rodada quando detecta necessidade de mudança. A decisão aplicada vem dos votos das réplicas.

Replicação, falhas e segurança

A votação usa o mesmo conjunto de entradas e identifica cada controlador por uma chave própria.

Replicação persistente

Os quatro controladores recebem a rodada independentemente e armazenam entrada e voto em arquivos SQLite distintos. O commit precede o envio do voto. Se a mesma rodada for entregue de novo, a réplica reutiliza o registro. Uma réplica que ficou offline pode perder rodadas. A demonstração cobre os registros recebidos por cada processo.

Queda durante a execução

O teste faz C1 encerrar após gravar sua decisão e antes de responder. C2, C3 e C4 formam quórum e o cliente recebe OK após o ACK do atuador. O middleware espera até 800 ms pelos votos. Com menos de três respostas iguais, retorna NO_QUORUM e preserva o último estado aplicado.

Respostas divergentes e falsificação

HMAC-SHA256 autentica o conteúdo de cada mensagem. O middleware verifica o ID da rodada, o tipo do voto e a chave do nó, contando um voto por identidade. Uma réplica com sua chave pode emitir um voto errado, mas três réplicas corretas ainda concordam. Uma tentativa de se passar por outro nó falha na verificação da chave.

Entrega e idempotência

MQTT QoS 1 pode repetir uma entrega. O atuador grava uma única aplicação por ID e o middleware devolve o recibo já registrado se a operação for repetida com o mesmo modo. Se faltar ACK, o cliente recebe UNKNOWN_OUTCOME e pode repetir com o mesmo ID.

Modelo de falha

O protótipo pressupõe um middleware confiável e serializa as operações. Broker e atuador são únicos. A rede local usa MQTT sem TLS e as chaves pressupõem isolamento de processos. Em caso de perda de quórum, a bomba simulada conserva o estado anterior, inclusive se estiver ligada.

Resultados e reprodução

Verificação em Windows com Python 3.12, broker MQTT local e processos reais separados.

Normal e repetição

Quatro votos a favor, aplicação confirmada e recibo reutilizado. O atuador recuperou seu estado após reinício.

Queda em plena rodada

C1 encerrou com código 17. C2, C3 e C4 votaram a favor e o cliente recebeu OK.

Divergência e identidade falsificada

O voto contrário válido de C1 ficou em minoria. A tentativa de forjar C2 foi descartada. Nos dois casos houve aplicação.

Perda de quórum e modo automático

Uma queda e um voto contrário resultaram em NO_QUORUM, com zero aplicações. Em AUTO a bomba ligou no nível alto e desligou no nível baixo.

Como executar

Instale as dependências com **python -m pip install -r requirements.txt**. No Windows, use **python demo.py --split** para abrir o Publisher e o Monitor. Use **python demo.py --verify** para reproduzir os cenários e **python -m unittest -v** para executar os três testes unitários.

Código-fonte e referências

GitHub · Vitorhrm/estacao-enchentes-sd (repositório público)
OASIS · MQTT 3.1.1, seções 3.3 e 4.3
Eclipse · Paho MQTT Python Client